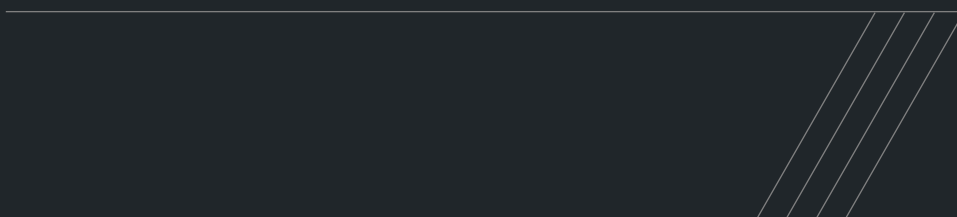


INNOVARE SUITE

REACTJS

Material inicial



FULL STACK DEVELOPER JUNIOR

REACTJS

Instalación de NodeJs

Aunque ReactJS es, simplemente, una librería que debemos incluir en nuestro proyecto, durante el proceso de creación de nuestra aplicación, esta se va a ejecutar en un servidor de NodeJS, y utilizaremos algunos comandos de NodeJS para instalar ciertos módulos. Por lo que debemos tenerlo instalado en nuestra PC.

¿Qué es NODEJS?

Es una tecnología que nos permite programar del lado del servidor mediante código JavaScript.

Utilizando comandos de NodeJs, podemos crear carpetas, ficheros, desplegar un servidor web, actualizar los módulos de nuestra aplicación, etc.

En el desarrollo de aplicaciones con ReactJs, usaremos NodeJs para ir lanzando en un servidor web nuestra aplicación a medida que la vamos desarrollando, y para instalar de forma cómoda los módulos que vayamos necesitando.

(Para la instalación de NodeJS seguir los pasos del material brindado en "Herramientas para el curso" (fecha: 07/07)).

Create React App – Creación de nuestra primera aplicación

La mejor alternativa para dar los primeros pasos con React es usar el paquete create-react-app. Lo que nos ahorrará muchos pasos de configuración inicial de un proyecto.

Cuando comenzamos a programar una nueva aplicación con React, no es habitual partir de una carpeta vacía e ir creando dentro la estructura de carpetas y ficheros necesarios.

Como estos primeros pasos (montar la estructura de la aplicación) suelen ser siempre los mismos, lo normal es partir de una plantilla como la que se mostrará a continuación.

Utilizaremos el comando **create-react-app**, que mostrará la estructura de ficheros y la configuración básica por nosotros.

- Lo primero que debemos hacer es crear una carpeta con el nombre de nuestra aplicación (sin caracteres especiales, ni espacios en blanco).
- Abrimos la carpeta recién creada con VSCode.
- Abrimos la terminal y escribimos ***npx create-react-app <nombreapp>***:

- Este comando es un comando de NodeJs, comienza con **npx** que es el acrónimo de Node Package Execute.
- El comando **create-react-app** corresponde a la acción que vamos a realizar. Es el encargado de descargar de Internet los archivos necesarios para montar nuestro proyecto y la estructura de carpetas de una aplicación que podremos ejecutar y que nos servirá de base para nuestros desarrollos.
- La palabra **<nombreapp>** hace referencia al nombre de la aplicación que vamos a crear. Este nombre será, además, el nombre de la carpeta en la que se va a crear y que contendrá el código fuente de dicha aplicación. También podemos especificar un nombre diferente, que creará otra carpeta dentro de la carpeta principal.

Estructura de la aplicación

En la estructura de carpetas que se acaba de crear hay varios archivos y carpetas. Un repaso por cada uno:

- **src:** en esta carpeta están la mayoría de los ficheros cuyo código fuente vamos a programar. Se puede decir que esta carpeta contiene el código fuente de nuestra aplicación.
- **src/App.js:** este es por el momento nuestro archivo más importante. Es el componente de inicio de nuestra aplicación.
- **src/App.css:** es una hoja de estilos cargada desde App.js mediante la siguiente línea de código: `import './App.css';`
- **package.json:** este archivo es propio de un proyecto hecho con **NodeJS**.

Tiene las siguientes características:

Contiene el nombre y la versión del proyecto.

Los scripts del proyecto.

Sirve para administrar las dependencias. Puede añadir más dependencias instalando los paquetes con el comando: `npm i nombre del paquete` (cuando se ejecuta el comando `npm i`, lo que hace es instalar todos los módulos referenciados en el `package.json`).

- **node_modules:** aquí están las librerías externas que usa el proyecto. Habitualmente, cuando uno quiere mandar el proyecto a otra persona o subirlo a una página como `github.com`, se debe evitar enviar esta carpeta, ya que es lo que más pesa, y puede cargarse cómodamente escribiendo `npm i` desde la terminal.
- **public:** en esta carpeta podemos almacenar los assets del proyecto, es decir, archivos de vídeo, audio, imágenes, etc. Lo almacenado en ella será

traspasado a la carpeta build cuando compilemos la aplicación sin ningún tipo de procesamiento.

Hasta aquí los más importantes por el momento, los demás archivos se pueden eliminar para mantener más limpio nuestro espacio. Al eliminar estos archivos innecesarios también deberemos eliminar las referencias a ellos para que no genere errores al ejecutar.

*EL EJECUTOR DE PAQUETES npx: desde la versión 5.2.0 de npm, npx está preinstalado con npm.
npx es también una herramienta CLI (Interfaz de línea de comandos) cuyo propósito es facilitar la instalación y la gestión de las dependencias alojadas en el registro npm.
Ahora es muy fácil ejecutar cualquier tipo de ejecutable basado en Node.js que normalmente se instalaría a través de npm.*

El ejecutor de paquetes **npx**: desde la versión 5.2.0 de **npm**, **npx** está preinstalado con **npm**.

npx es también una herramienta CLI (Interfaz de línea de comandos) cuyo propósito es facilitar la instalación y la gestión de las dependencias alojadas en el registro **npm**.

Ahora es muy fácil ejecutar cualquier tipo de ejecutable basado en Node.js que normalmente se instalaría a través de npm. Hola mundo

El más pequeño de los ejemplos de React se e así:

```
ReactDOM.render (
  <h1>Hello, world!</h1>,
  document.getElementById('root')
);
```

Esto muestra un encabezado con el texto "Hello, world!" en la página.

Presentando JSX

Consideremos la declaración de esta variable:

```
const element = <h1>Hello, world!</h1>;
```

Esta curiosa sintaxis de etiquetas no es ni un string ni HTML. Es JSX, y es una extensión de la sintaxis de JavaScript. Se recomienda su uso con React para describir cómo debería ser la interfaz de usuario.

JSX produce “elementos” de React. En breve se verá como renderizarlos en el DOM.

Ahora, con React no debemos usar `document`, `querySelector`, ni `document.getElementById`, ni cualquier otro tipo de acceso al DOM. En lugar de esto, escribiremos la variable intercalada en el código HTML y, podremos mostrar directamente su valor.

Ejemplo: el siguiente código no utiliza JSX. Lo que hace es asignar a una etiqueta `h1` el valor de una constante.

```
<h1></h1>
<script>
  const valor = 5;
  document.querySelector('h1').innerHTML = valor;
</script>
```

El código equivalente al anterior utilizando JSX sería algo así:

```
const valor = 5;
const código = <h1>{valor}</h1>;
```

Lo que se hizo fue traducir de JavaScript normal a código JSX. React hace justo el proceso inverso. Mediante un transpilador, traduce el código escrito por nosotros en formato JSX a un código JavaScript que puede interpretar el navegador web.

Insertando expresiones en JSX

En el ejemplo a continuación, se declara una variable llamada `name` y luego la usamos dentro del JSX envolviéndola dentro de llaves:

```
const name = 'Josh Perez';
const element = <h1>Hello, {name}</h1>;
ReactDOM.render (
  element,
  document.getElementById('root')
);
```

Podemos poner cualquier expresión de JavaScript dentro de las llaves en JSX.

A continuación, insertamos el resultado de llamar la función de JavaScript, `formatName(user)`, dentro de un elemento `<h1>`.

```
function formatName(user) {
  return user.firstName + ' ' + user.lastName;
}
```

```
const user = {
  firstName: 'Harper',
  lastName: 'Perez'
};
```

```
const element = (
  <h1>Hello, {formatName(user)} ! </h1>
);
```

```
ReactDOM.render (
  element,
  document.getElementById('root')
);
```

JSX como una expresión

Después de compilarse, las expresiones JSX se convierten en llamadas a funciones JavaScript regulares y se evalúan en objetos JavaScript.

Esto significa que podemos usar JSX dentro de declaraciones if y bucles for, asignarlo a variables, aceptarlo como argumento, y retornarlo desde dentro de funciones:

```
function getGreeting(user) {
  if (user) {
    return <h1>Hello, {formatName(user)} !</h1>;
    return <h1>Hello, Stranger.</h1>; }
}
```

Especificando atributos

Podemos utilizar comillas para especificar strings literales como atributos:

```
const element = <img src={user.avatarUrl}></img>;
```

Si una etiqueta está vacía, podemos cerrarla inmediatamente con `/>`, como en XML:

```
const element = <img src={user.avatarUrl} />;
```

Las etiquetas de JS pueden contener hijos:

```
const element = (
  <div>
    <h1>Hello!</h1>
    <h2>Good to see you here.</h2>
  </div>
);
```

JSX representa objetos

Estos objetos son llamados “Elementos de React”. React lee estos objetos y los usa para construir el DOM.

Supongamos que tenemos el siguiente elemento:

```
const elemento = (<h1 className = 'saludo' >Hello, world!</h1>);
```

Este elemento va a ser idéntico al siguiente elemento:

```
const elemento = React.createElement('h1',
{className = 'saludo'},
Hello, world!);
```

Renderizando elementos

Los elementos son los bloques más pequeños de las aplicaciones de React.

Un elemento describe lo que queremos ver en pantalla:

```
const element = <h1>Hello, world</h1>;
```

A diferencia de los elementos del DOM de los navegadores, los elementos de React son objetos planos, y su creación es de bajo costo. React DOM se encarga de actualizar el DOM para igualar los elementos de React.

NOTA: podríamos confundir los elementos con el concepto de “componentes”. Los elementos son los que ‘constituyen’ los componentes.

Digamos que hay un <div> en alguna parte de nuestro archivo HTML:

```
<div id='root' ></div>
```

Lo llamamos un nodo “raíz” porque todo lo que esté dentro de él será manejado por React DOM.

Las aplicaciones construidas solamente con React usualmente tienen un único nodo raíz en el DOM. Dado el caso que estemos integrando React con una aplicación existente, podemos tener tantos nodos raíz del DOM aislados como queramos.

Para renderizar un elemento con un nodo raíz del DOM, pasamos ambos a ReactDOM.render():

```
const element = <h1>Hello, world</h1>;
ReactDOM.render(element, document.getElementById('root'));
```

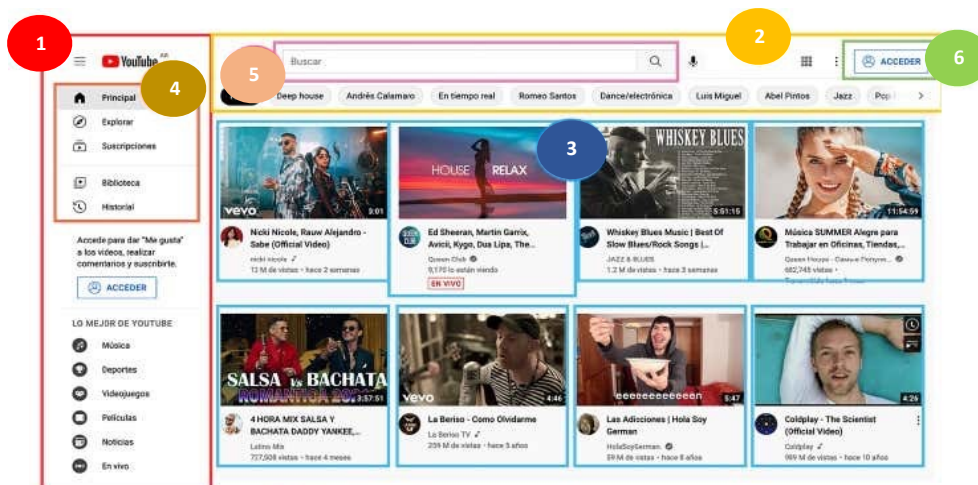
Esto muestra "Hello, world" en la página.

Actualizando el elemento

React solo actualiza lo que sea necesario. ReactDOM compara el elemento y sus hijos con el elemento anterior, y solo aplica las actualizaciones del DOM que son necesarias para que éste se encuentre en el estado deseado.

COMPONENTES

Como se mencionó, las aplicaciones React se construyen mediante componentes, los cuales son elementos independientes y pueden ser reutilizados, además, describen como tienen que visualizarse y cómo tienen que comportarse. Los componentes permiten separar la interfaz de usuario en piezas independientes.



Dentro de una página como Youtube, por ejemplo, podemos marcar varios componentes:

1) SideBar

2) Header

- 3) **Cada carta en su propio componente.** También dentro de cada componente existen otros componentes.
- 4) **Los botones Principal, Explorar, Suscripciones, Biblioteca e Historial.** Cada uno de esos botones es un componente, que vive dentro del componente Sidebar.
- 5) **El buscador es un componente dentro del Header.**
- 6) **El botón de login es otro componente dentro del Header.**

Y podríamos nombrar muchos componentes más dentro de esta página. Cada aplicación React comienza con un componente raíz y se compone de muchos **componentes en una formación de árbol**. Los componentes en React son **funciones** que dejan la UI (interfaz de usuario) basada en la data (apoyo y estado) que recibe.

Conceptualmente, los componentes son como las funciones de JavaScript. Aceptan entradas arbitrarias (llamadas props) y devuelven a React **elementos que describen lo que debe aparecer o se debe ver en la pantalla**.

Los props son las propiedades o información que viaja de un componente padre a un hijo. Más adelante se explicará en detalle.

Existen dos tipos de componentes en React:

Componentes funcionales: Solo tienen propiedades.

- El nombre del componente debe comenzar con mayúscula.
- Un componente debe devolver código JSX (markup) o null.

Componentes de clase: Tienen propiedades, ciclos de vida y estado. La forma más sencilla de definir un componente es escribir una función de JavaScript:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

Esta función es un componente de React válido porque acepta un solo argumento de objeto props con datos y devuelve un elemento de React.

Llamamos a dichos componentes funcionales porque literalmente son funciones JS.

El nombre del componente debe comenzar con mayúscula.

Un componente debe devolver código JSX (markup) o **null**. También podemos utilizar una clase de ES6 para definir un componente:

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

¿Para qué sirve un componente?

Crear nuestros propios componentes nos será útil para poder reutilizarlos en nuestras aplicaciones. También podremos utilizar componentes definidos por otras personas, lo que nos reducirá muchísimo el tiempo de desarrollo.

Podemos crear componentes para popups, consulta de una API del tiempo meteorológico, mostrar un reloj...

Ejemplos de uso de componentes

La función App dentro del archivo App.js, es un componente, ya que devuelve código JSX.

En el siguiente ejemplo se declara un nuevo componente llamado

MyComponent, dentro del archivo App.js. Este componente devuelve código JSX, concretamente un div con un texto:

```
const MyComponent = () => <div>Hola</div>;  
const App = () => {  
  return (  
    <div>  
      <MyComponent />  
    </div>  
  );  
}  
export default App;
```

Como se puede observar, una vez declarado el componente MyComponent, se inserta en la página web como si fuese una etiqueta HTML normal.

Cada vez que el componente se renderiza, sus funciones se vuelven a crear, por lo que cuantas más funciones podamos definir fuera del mismo, mejor.

Creación de un componente en un archivo externo

Ahora, desde App.js importaremos un componente que se encuentra en otro archivo independiente.

El resultado de este ejemplo es igual al del ejemplo anterior.

```
import MyComponent from './components/MyComponent'
```

```
const App = () => (  
  <div>  
    <MyComponent />  
  </div>  
);  
export default App;
```

Finalmente, sólo falta definir el código fuente del componente MyComponent:

```
const MyComponent = () => <div>Hola</div>  
  
export default MyComponent;
```

Otro ejemplo:

```
function MyButton() {  
  return (  
    <button>Soy un botón</button>  
  );  
}
```

En este código se ha declarado MyButton, podemos anidarlo en otro componente:

```
export default function MyApp() {  
  return (  
    <div>  
      <h1>Bienvenido a mi aplicación</h1>  
      <MyButton />  
    </div>  
  );  
}
```

Como se ve, el componente MyButton empieza con mayúscula. Así es como sabemos que es un componente de React, mientras que las etiquetas HTML deben estar en minúsculas.

El resultado del código será la leyenda "Bienvenido a mi aplicación" y un botón debajo con la leyenda "Soy un botón".

Las palabras clave export default especifican el componente principal en el archivo. Para saber más sobre export la siguiente referencia es muy recomendable MDN.

RECORDAR!!!
*JSX es más estricto que HTML. Debemos cerrar etiquetas como
. El componente creado tampoco puede devolver múltiples etiquetas de JSX. Se las debe envolver en un padre compartido, como <div> ... </div> o en un envoltorio vacío <>...</>.*

Añadir estilos

En React, para especificar una clase de CSS utilizamos className. Funciona de la misma forma que el atributo class de HTML:

```
<img className = "avatar" />
```

Luego escribimos las reglas CSS para esa clase en un archivo CSS aparte:

```
/* En el CSS */
.avatar {
  border-radius: 50%;
}
```

React no prescribe como añadir los archivos CSS. En el caso más simple, añadimos una etiqueta <link> a nuestro HTML, que es como lo hicimos hasta ahora.

PROPS

Sintácticamente, una prop es similar a los atributos de HTML: nombre del atributo, signo de igual y, entre comillas, el valor del atributo.

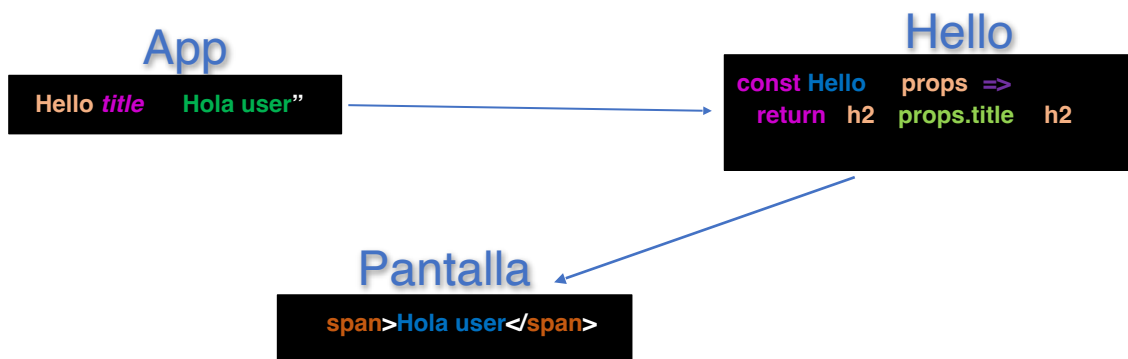
En el siguiente ejemplo, la prop title pasará un valor del componente App al componente Hello:

```
const App = () => (
  <Hello title = "Hola user" />
);

const Hello = (props) => {
  return <h2>{props.title}</h2>
};

export default App;
```

Dentro del componente App se ha cargado un componente llamado Hello. Este componente tiene un atributo llamado title (una prop), cuyo valor es "Hola user". Lo que se hace con esta prop es pasar valores del componente padre (App) al componente hijo (Hello).



Estados

Los componentes a menudo necesitan cambiar lo que se muestra en pantalla como resultado de una interacción. Escribir dentro de un formulario debería actualizar el campo de texto, hacer click en **–siguiente–** en un carrusel de imágenes debería cambiar la imagen que es mostrada, hacer click en un botón para comprar un producto debería actualizar el carrito de compras. En estos ejemplos los componentes deben recordar cosas: el campo de texto, la imagen actual, el carrito de compras. Este tipo de memoria de los componentes se conoce como **estado** en **React**.

Un estado es una variable de cuyos cambios de valor **React** es notificado. Cuando cambiamos el valor de una variable normal, no ocurre nada en particular. Sin embargo, cuando cambiamos el valor de un estado, React es notificado y puede actualizar el valor de dicha variable automáticamente en la pantalla, o incluso ejecutar funciones.

El estado es un objeto que se agrega como propiedad en los componentes de la clase.

Ejemplo:

```
class Hello extends React.Component {  
  state = {  
    name: "James"  
  }  
  
  render() {  
    return <h1>Hello {this.state.name}</h1>;  
  }  
}
```

Como se puede ver, el estado es solo un objeto simple que contiene pares **clave: valor**. De manera similar a los **props**, se puede acceder a los valores usando `this.state`.

Ahora, cuando el componente se procesa, el estado se inicializa con el valor dado y habrá un encabezado que dice "Hola James".

El estado no debe modificarse directamente. En cambio, React proporciona un método `setState()`, que puede usarse para modificarlo.

Ejemplo:

```
this.setState({  
  name: "James",  
  age: 25  
});
```

Se debe pasar un objeto con los nuevos pares **clave: valor** al método **setState**.

¿Por qué usar `setState`, en lugar de simplemente cambiar los valores de las propiedades del objeto directamente?

Porque una de las características más útiles de **React** es que cuando se llama a `setState`, éste vuelve a **renderizar** automáticamente el componente afectado con el nuevo estado. Cuando el estado cambia usando el método `setState`, React se informa e inmediatamente vuelve a renderizar el componente con el estado actualizado.

En el siguiente ejemplo visualizaremos mejor cómo funciona el estado, crearemos una aplicación de contador que incrementa cada vez que se hace clic en un botón.

Crearemos el componente **Contador**, que incluye el contador y un botón:

```

class Contador extends React.Component {
  state = {
    counter: 0
  }
  render() {
    return <div>
      <p>{this.state.counter}</p>
      <button>Incremento</button>
    </div>;
  }
}

```

Se ha inicializado el contador al valor 0 en el estado.

Ahora, es necesario agregar un controlador de eventos clic al botón e incrementar el contador en el estado.

```

class Contador extends React.Component {
  state = {
    counter: 0
  }
  increment = () => {
    this.setState({
      counter: this.state.counter+1});
  }
  render() {
    return <div>
      <p>{this.state.counter}</p>
      <button onClick={this.increment}>Incremento</button>
    </div>;
  }
}

```

El evento onClick llama a la función de incremento del componente, que usa setState para cambiar el valor del contador. Cuando se cambia el estado, React activa automáticamente una nueva renderización del componente.

props vs estado

A modo de resumen, listaremos a continuación las principales diferencias entre props y estado:

- Usamos props para pasar datos a los componentes.
- Los componentes utilizan el estado para gestionar sus datos.
- Los props son de solo lectura y no se pueden modificar.

- El estado puede ser modificado por su componente usando el método `setState()`.
- El método `setState()` da como resultado volver a renderizar el componente afectado.

*Los componentes que tienen estado se denominan **con estado**, mientras que los componentes que no utilizan estado se denominan **sin estado**.*

HOOKS

La versión anterior de React permitía usar el estado solo con componentes de clase. En iteraciones recientes de React se introdujo una nueva característica llamada **Hooks**, que permite usar el estado de componentes funcionales.

Primero, es necesario importar el Hook `useState`:

```
import React, {useState} from 'react';
```

`useState` devuelve un par, el valor del estado actual y una función que le permite cambiar el estado.

`useState` toma un argumento, que es el valor inicial del estado.

Ejemplo:

```
function Hello() {  
  const [name, setName] = useState("David");  
  return <h1>Hello {name}</h1>;  
}
```

En el ejemplo, se crea una variable de estado de nombre y una función `setName`. La sintaxis de corchetes se llama **desestructuración** de un array y permite leer valores de un array. Asigna la variable `name` al valor del estado actual y `setName` a la función que permite cambiar el estado. Podemos nombrar las variables de cualquier forma pero lo convencional es nombrar estas dos variables como `const [algo, serAlgo]`. El array devuelto por `useState` siempre contará con dos valores.

Luego se pasa "David" como valor inicial de la variable `name` para `useState()`.

Podemos crear múltiples variables de estado con sus correspondientes métodos establecidos. Simplemente usamos declaraciones separadas para cada variable con el Hook `useState`.

Las funciones que comienzan con `use` se llaman Hooks. `useState` es un Hook nativo dentro de React. Existen otros Hooks nativos en React y también

podemos escribir nuestros propios **Hooks** mediante la combinación de otros existentes.

Los Hooks son más restrictivos que las funciones regulares. Solo podemos llamar a los Hooks en el primer nivel de nuestros componentes (u otros Hooks). Si queremos utilizar `useState` en una condicional o en un bucle, debemos extraer un nuevo componente y colocarlo ahí. Los Hooks son funciones, pero es útil pensar en ellos como declaraciones independientes de las necesidades de nuestro componente.

Las funciones de React se usan en la parte superior del componente de manera similar a cómo se importan módulos en la parte superior de un archivo.

Reescribiremos la aplicación del contador realizada más arriba usando un componente funcional y Hooks:

```
function Contador() {  
  const [counter, setCounter] = useState(0);  
  function increment() {  
    setCounter(counter + 1);  
  }  
  return <div>  
    <p>{counter}</p>  
    <button onClick = {increment}>  
      Incremento  
    </button>  
  </div>;  
}
```

Como se puede ver, en comparación con el componente de clase, el código es mucho más corto y más fácil de leer y comprender. Esta fue una de las razones por las que el equipo de **React** creó Hooks.

¡RECORDAR!

*Los Hooks sólo se pueden utilizar dentro de componentes funcionales.
Los Hooks son funciones que permiten 'conectarse' a las características de React desde los componentes de la función.*

Anatomía del `useState`

Cuando llamamos al `useState`, le estamos diciendo a React que debe recordar algo:

```
const [counter, setCounter] = useState(0);
```

En este caso, queremos que React recuerde el counter.

El único argumento para **useState** es el valor inicial de su variable de estado.

En este ejemplo, el valor inicial de counter se establece en 0 con `useState(0)`.

Cada vez que el componente se renderiza, el **useState** devuelve un array que contiene dos valores:

- a- La variable de estado (counter) con el valor que almacenamos.
- b- La función que establece el estado (setCounter) que puede actualizar la variable de estado y alertar a React para que renderice el componente nuevamente.

- Así es como sucede en acción:

- Nuestro componente se renderiza por primera vez. Debido a que pasamos 0 a `useState` como valor inicial para counter, esto devolverá `[0, setCounter]`. React recuerda que 0 es el último valor de estado.
- Actualizamos el estado. Cuando un usuario hace clic en el botón, llama a `setCounter(counter + 1)`. counter es 0, por lo tanto es `setCounter(1)`. Esto le dice a React que recuerde que counter es 1 ahora y ejecuta otro renderizado.
- El componente se renderiza por segunda vez. React todavía ve `useState(0)`, pero debido a que React recuerda que estableció counter en 1, devuelve `[1, setCounter]` en su lugar.
- ¡Y así sucesivamente!

Hooks integrados de React

Sólo nombraremos los Hooks, para más información de cada uno use la siguiente referencia a la página oficial de [React](#).

Hooks de estado:

- `useState`: declara una variable de estado que podemos actualizar directamente.
- `useReducer`: declara una variable de estado con la lógica de actualización dentro de una función reductora.

Hooks de contexto: permiten a un componente recibir información de padres lejanos sin pasarlas como props.

- `useContext`: lee y se suscribe a un contexto.

Hooks de refs: las refs permiten a un componente mantener alguna información que no es utilizada para el renderizado, como un nodo del DOM o el ID de un timeout. Son útiles cuando necesitamos trabajar con sistemas distintos de React, como las APIs integradas del navegador.

- `useRef`: declara una ref. Puede contener cualquier valor, pero la mayoría de las veces se utiliza para contener un nodo del DOM.
- `useImperativeHandle`: permite personalizar la ref expuesta por nuestro componente. Esto rara vez se usa.

Hooks de efecto: permite a un componente conectarse y sincronizarse con sistemas externos.

- `useEffect`: conecta un componente a un sistema externo.
- `useLayoutEffect`
- `useInsertionEffect`

Hooks de rendimiento: una forma común de optimizar el rendimiento del renderizado es evitar trabajo innecesario.

- `useMemo`: permite guardar en caché los resultados de un cálculo costoso.
- `useCallback`: permite guardar en caché una función definida antes de pasarla a un componente optimizado.

Para priorizar el renderizado:

- `useTransition`
- `useDeferredValue`

Otros Hooks: son mayormente usados para autores de bibliotecas y no son comúnmente usadas en el código de la aplicación.

- `useDebugValue`
- `useId`
- `useSyncExternalStore`

